

Самостоятельная работа 1 Введение в TypeScript: базовые типы, настройка  
проекта, преимущества типизации

Дисциплина: Разработка и оптимизация веб-приложений

Выполнили:

Гаврилов Н.С.

Лялин И.М

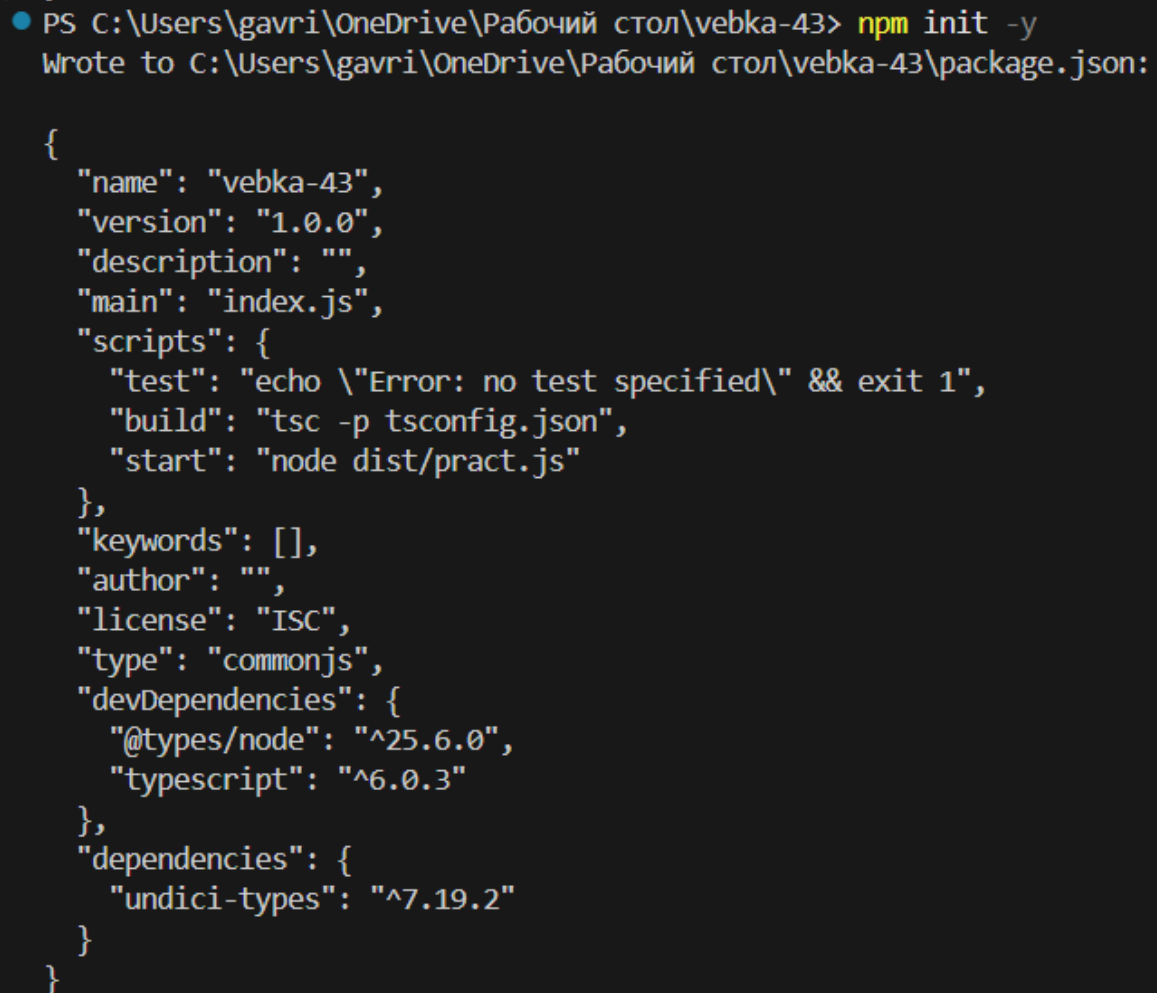
Проверила:

Зиберева А.А.

## Шаг 1. Инициализация проекта и создание .gitignore

«npm init -y» - команда, создающая файл package.json с настройками «по умолчанию»

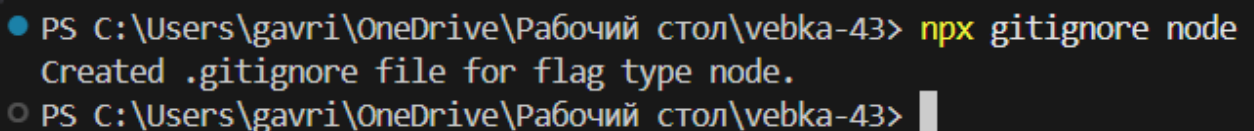
«npx gitignore node» - команда, генерирующая .gitignore, исключающий из репозитория папки node\_modules, dist, логи и др.



```
PS C:\Users\gavri\OneDrive\Рабочий стол\vebka-43> npm init -y
Wrote to C:\Users\gavri\OneDrive\Рабочий стол\vebka-43\package.json:

{
  "name": "vebka-43",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "build": "tsc -p tsconfig.json",
    "start": "node dist/pract.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "devDependencies": {
    "@types/node": "^25.6.0",
    "typescript": "^6.0.3"
  },
  "dependencies": {
    "undici-types": "^7.19.2"
  }
}
```

Рисунок 1-результат работы кода



```
PS C:\Users\gavri\OneDrive\Рабочий стол\vebka-43> npx gitignore node
Created .gitignore file for flag type node.
PS C:\Users\gavri\OneDrive\Рабочий стол\vebka-43> █
```

Рисунок 2-результат работы кода

## Шаг 2. Установка TypeScript и типов Node.js

«npm i -D typescript @types/node»- npm i -D устанавливает пакеты в раздел devDependencies (нужны только на этапе разработки), typescript — это компилятор,

превращающий TypeScript-код в обычный JavaScript, а @types/node содержит описания типов для всех встроенных модулей Node.js (например, console, fs, path), чтобы TypeScript понимал их структуру и мог проверять корректность их использования.

```
PS C:\Users\gavri\OneDrive\Рабочий стол\vebka-43> npm i -D typescript @types/node

changed 2 packages, and audited 4 packages in 6s

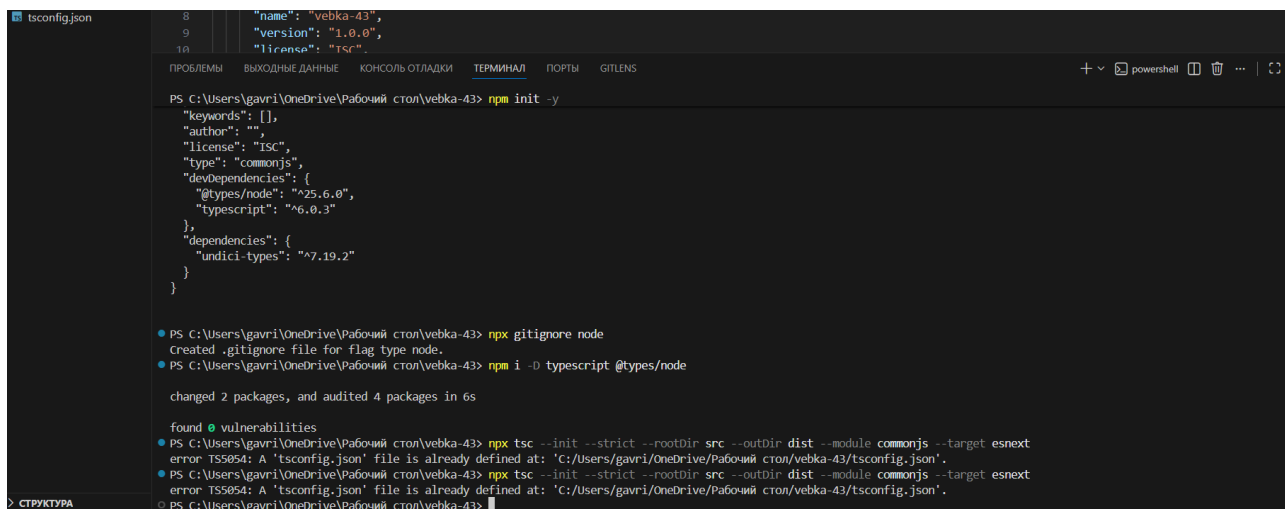
found 0 vulnerabilities
PS C:\Users\gavri\OneDrive\Рабочий стол\vebka-43>
```

Рисунок 3-результат работы команды

### Шаг 3. Генерация tsconfig.json

«npx tsc --init --strict --rootDir src --outDir dist --module commonjs --target esnext»-:

«npx tsc --init» создаёт конфигурационный файл TypeScript; флаг --strict включает все строгие проверки типов (например, noImplicitAny, strictNullChecks); --rootDir src указывает, что исходные .ts файлы находятся в папке src; --outDir dist задаёт папку для скомпилированных .js файлов; --module commonjs выбирает модульную систему CommonJS, которая используется в Node.js; --target esnext говорит компилятору генерировать современный JavaScript-код без транспилиции новых возможностей языка.



```
tsconfig.json
{
  "name": "vebka-43",
  "version": "1.0.0",
  "license": "ISC",
  "keywords": [],
  "author": "",
  "type": "commonjs",
  "devDependencies": {
    "@types/node": "^25.6.0",
    "typescript": "^6.0.3"
  },
  "dependencies": {
    "undici-types": "^7.19.2"
  }
}

PS C:\Users\gavri\OneDrive\Рабочий стол\vebka-43> npm init -y
Created tsconfig.json file for flag type node.
PS C:\Users\gavri\OneDrive\Рабочий стол\vebka-43> npm i -D typescript @types/node
changed 2 packages, and audited 4 packages in 6s
found 0 vulnerabilities
PS C:\Users\gavri\OneDrive\Рабочий стол\vebka-43> npx tsc --init --strict --rootDir src --outDir dist --module commonjs --target esnext
error TS5054: A 'tsconfig.json' file is already defined at: 'C:\Users\gavri\OneDrive\Рабочий стол\vebka-43\tsconfig.json'.
PS C:\Users\gavri\OneDrive\Рабочий стол\vebka-43> npx tsc --init --strict --rootDir src --outDir dist --module commonjs --target esnext
error TS5054: A 'tsconfig.json' file is already defined at: 'C:\Users\gavri\OneDrive\Рабочий стол\vebka-43\tsconfig.json'.
PS C:\Users\gavri\OneDrive\Рабочий стол\vebka-43>
```

Рисунок 4-результат работы команды

## Шаг 4: Создание папки src и файла pract.ts

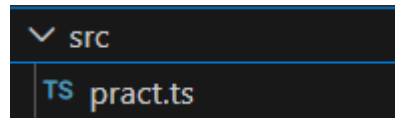


Рисунок 5-создание папки src и в ней файл pract.ts

## Шаг 5. Реализация кода в pract.ts

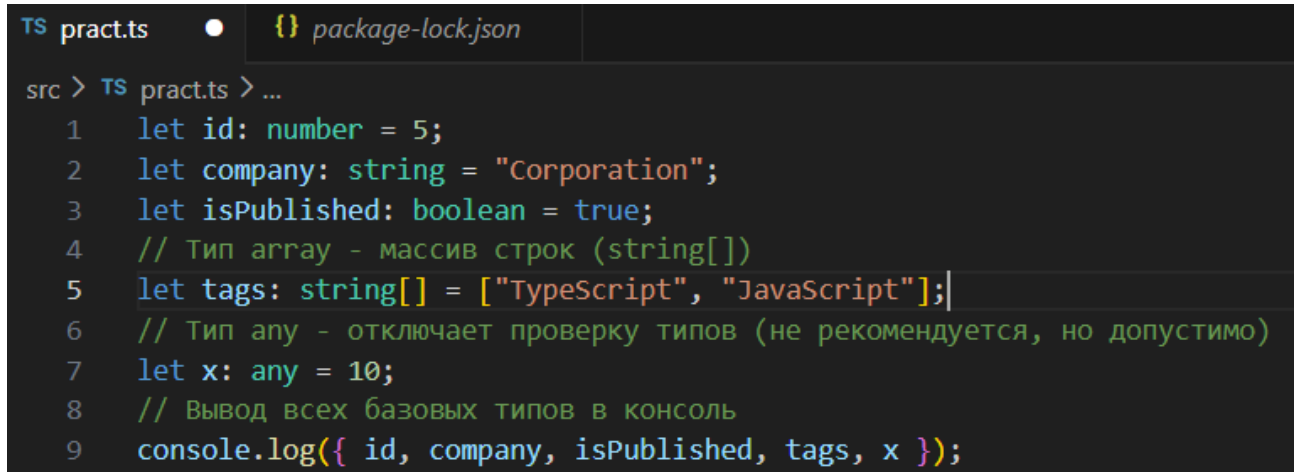


Рисунок 5.1 — Базовые типы:

В этой части кода объявляются переменные с явными аннотациями типов `string`, `number`, `boolean`, `any`, `null` и `undefined`; TypeScript проверяет, что каждое присвоенное значение соответствует заявленному типу, и если попытаться присвоить, например, число в переменную типа `string`, компилятор выдаст ошибку ещё до запуска программы.

```

// interface - описывает структуру объекта
interface User {
  id: number;
  name: string;
  age?: number; // ? означает опциональное поле (может отсутствовать)
  greet: (message: string) => void; // метод с типизированным параметром
}

// Создаём объект, соответствующий интерфейсу User
const user: User = {
  id: 1,
  name: "Anna",
  // age отсутствует - это допустимо, т.к. age опциональный
  greet: (message: string) => console.log(message), // реализация метода
};

// Проверяем наличие опционального поля (выведет сообщение, т.к. age нет)
console.log(user.age === undefined ? "No age of the user" : user.age);
// Вызываем метод greet у объекта
user.greet("Hello from User.greet()");

// Типизированная функция: (a: string, b: string) => string
function concatValues(a: string, b: string): string {
  return a + " " + b;
}
console.log(concatValues("hello", "world"));

```

Рисунок 5.2- Интерфейсы, опциональные поля, типы функций

Здесь через ключевое слово `interface` описывается структура объекта `User` с обязательными полями `id` и `name` и опциональным полем `email` (знак `?` означает, что поля может не быть); также через интерфейс `MathOperation` описывается тип функции, которая принимает два числа и возвращает число, а переменная `add` получает этот тип и может содержать только функцию с соответствующей сигнатурой.

```

// Union использует символ | (значение может быть ИЛИ string ИЛИ number)
type Id = string | number;

// Функция принимает union тип (string ИЛИ number)
function printId(value: Id): void { // void - функция ничего не возвращает
  console.log(`ID is equal to ${value}`);
}

printId("ID 123");
printId(123);

```

Рисунок 5.3 — Объединение (union):

С помощью оператора `|` создаётся пользовательский тип `ID`, который может быть либо строкой (`string`), либо числом (`number`); функция `printId` принимает такой тип и может работать с обоими вариантами, а TypeScript следит, чтобы внутри

функции не выполнялись операции, недопустимые для одного из типов (например, методы строки для числа).

```
50 // Первый интерфейс
51 interface BusinessPartner {
52     name: string;
53     creditscore: number;
54 }
55
56 // Второй интерфейс
57 interface UserIdentity {
58     id: number;
59     email: string;
60 }
61
62 // Intersection использует символ & (объединяет ВСЕ поля из обоих интерфейсов)
63 type Employee = BusinessPartner & UserIdentity;
64
65 // Employee теперь имеет name, creditscore, id, email - все поля сразу
66 function signContract(employee: Employee): void {
67     console.log(
68         `Contract signed by ${employee.name}, email: ${employee.email}, creditscore: ${employee.creditscore}`
69     );
70 }
71
72 // Должны передать все поля из BusinessPartner и UserIdentity
73 signContract({
74     name: "Anna",
75     creditscore: 800,
76     id: 23,
77     email: "anna@gmail.com",
78 });
```

Рисунок 5.4 — Склейка двух интерфейсов (intersection):

Оператор & объединяет два интерфейса (Person и Employee) в один тип Staff; переменная worker должна содержать все поля из обоих интерфейсов — одновременно name из Person и salary из Employee, и TypeScript требует их обязательного присутствия при создании объекта.

```
80 // enum - набор именованных констант
81 enum LoginError {
82     Unauthorized = "unauthorized", // явно задаём строковые значения
83     NoUser = "no_user",
84     WrongCredentials = "wrong_credentials",
85     Internal = "internal",
86 }
87
88 // Функция обрабатывает различные типы ошибок на основе enum
89 function printLoginErrorMessage(error: LoginError): void {
90     switch (error) {
91         case LoginError.Unauthorized:
92             console.log("User not authorized");
93             return;
94         case LoginError.NoUser:
95             console.log("No user was found with that username");
96             return;
97         case LoginError.WrongCredentials:
98             console.log("Wrong credentials");
99             return;
100        default:
101            console.log("Internal error");
102            return;
103    }
104 }
105
106 // Передаём значения enum вместо "магических строк"
107 printLoginErrorMessage(LoginError.Unauthorized);
108 printLoginErrorMessage(LoginError.NoUser);
109 printLoginErrorMessage(LoginError.WrongCredentials);
110 printLoginErrorMessage(LoginError.Internal);
```

Рисунок 5.5 — Перечисление (enum):

enum Role создаёт набор именованных констант, где Role.Admin имеет значение "ADMIN", Role.User — "USER", а Role.Guest — "GUEST"; это удобно для задания ограниченного набора вариантов (роли пользователя, статусы заказа и т.п.), причём в отличие от простых строк, enum даёт автодополнение в IDE и централизованное управление значениями.

```
2 // <T> - generic (работает с любым типом, но сохраняет его)
3 class StorageContainer<T> {
4     // Массив элементов типа T
5     private contents: T[] = [];
6
7     // Добавляем элемент типа T
8     addItem(item: T): void {
9         this.contents.push(item);
10    }
11
12    // Получаем элемент по индексу (возвращает T или undefined)
13    getItem(index: number): T | undefined {
14        return this.contents[index];
15    }
16 }
17
18 // Создаём контейнер для строк (T = string)
19 const usernames = new StorageContainer<string>();
20 usernames.addItem("Anna");
21 usernames.addItem("Echo BR");
22 console.log(usernames.getItem(0));
23
24 // Создаём контейнер для чисел (T = number)
25 const friendCounts = new StorageContainer<number>();
26 friendCounts.addItem(23);
27 friendCounts.addItem(56);
28 console.log(friendCounts.getItem(1));
```

Рисунок 5.6 — Обобщённые типы (generics):

Функция `identity<T>` объявляет обобщённый параметр `T`, который фиксируется в момент вызова: `T` становится тем типом, который передан в аргументе; TypeScript сохраняет эту информацию на всём протяжении функции, поэтому результат имеет тот же тип, что и аргумент (`string → string`, `number → number`), обеспечивая типобезопасность без дублирования кода под каждый тип отдельно.

```

// readonly - поле можно прочесть, но нельзя изменить после создания
interface EmployeeReadOnly {
  readonly employeeId: number; // только для чтения
  name: string; // можно изменять
  readonly startDate: Date; // только для чтения
  department: string; // можно изменять
}

const emp: EmployeeReadOnly = {
  employeeId: 1,
  name: "Anna",
  startDate: new Date(),
  department: "Finance",
};

// Изменяем обычное поле
emp.name = "Jessica";
// emp.employeeId = 2; // ОШИБКА! Нельзя изменить readonly поле
// Выводим объект в консоль
console.log(emp);

```

Рисунок 5.7 — Модификатор типа «Только для чтения» (readonly):

В интерфейсе Point поля `x` и `y` помечены ключевым словом `readonly`, что означает невозможность их изменения после создания объекта; если в коде попытаться написать `p.x = 5`, TypeScript выдаст ошибку на этапе компиляции, защищая данные от случайных или нежелательных изменений.

## Шаг 6. Добавление скриптов в package.json

```

{} package.json > ...
1  {
2    "name": "vebka-43",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "build": "tsc -p tsconfig.json",
9      "start": "node dist/pract.js"
10   },
11   "keywords": [],
12   "author": "",
13   "license": "ISC",
14   "type": "commonjs",
15   "devDependencies": {
16     "@types/node": "^25.9.0",
17     "typescript": "^6.0.3"
18   },
19   "dependencies": {
20     "undici-types": "^7.19.2"
21   }
22 }

```

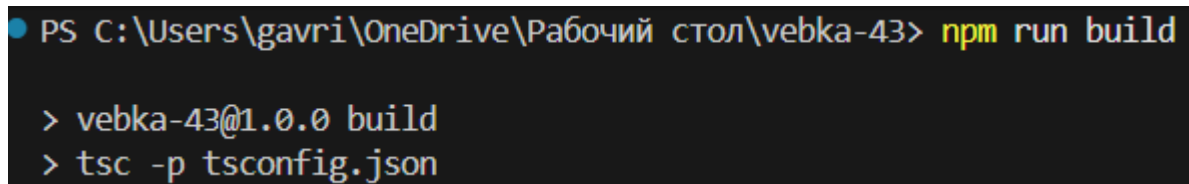
Рисунок 6-измененный скрипт

Добавленные строки в секцию "scripts": "build": "tsc -p tsconfig.json" и "start": "node dist/pract.js"



Скрипт `build` запускает компилятор TypeScript с указанием конкретного конфигурационного файла `tsconfig.json` (ключ `-p` означает «project»), что превращает весь код из `src` в JavaScript в папке `dist`; скрипт `start` выполняет скомпилированный файл `dist/pract.js` с помощью Node.js, позволяя увидеть результат работы программы.

## Шаг 7. Компиляция проекта



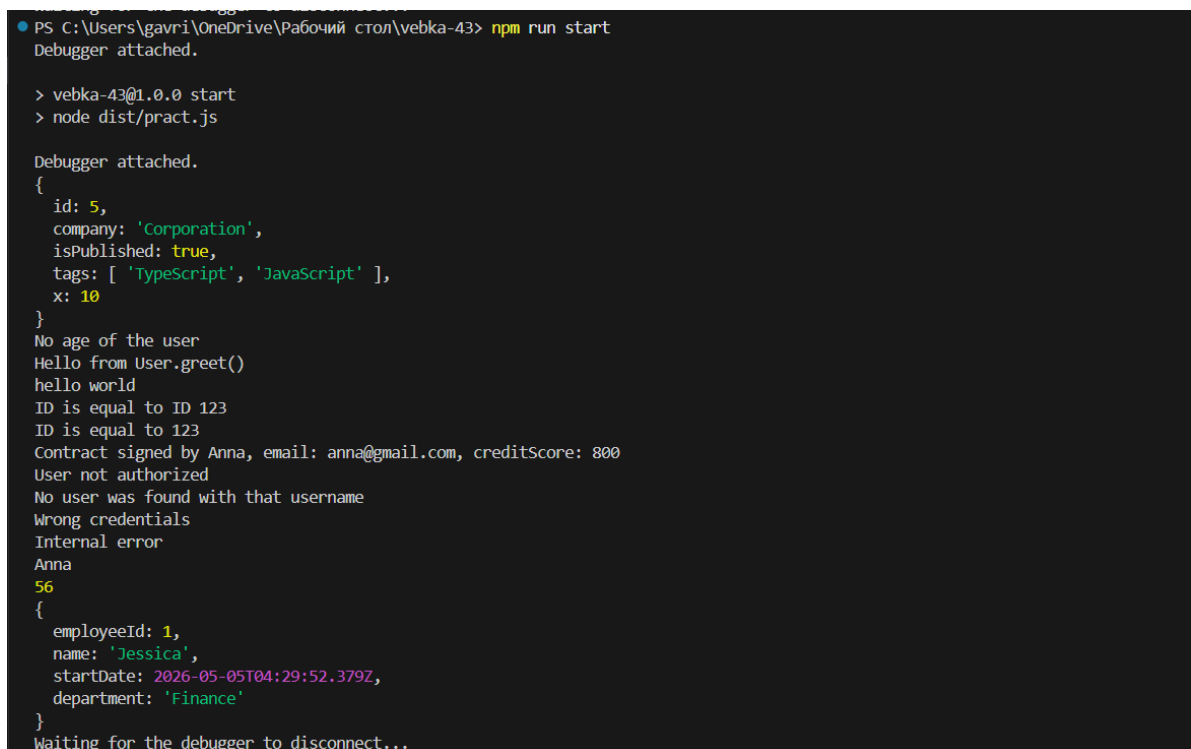
```
PS C:\Users\gavri\OneDrive\Рабочий стол\vebka-43> npm run build

> vebka-43@1.0.0 build
> tsc -p tsconfig.json
```

Рисунок 7- компиляция проекта

«`npm run build`»: данная команда вызывает скрипт `build` из `package.json`, который запускает компилятор `tsc`; компилятор читает `tsconfig.json`, находит все `.ts` файлы в `src`, выполняет проверку типов и, если ошибок нет, генерирует соответствующие `.js` файлы в папке `dist`, сохраняя структуру каталогов.

## Шаг 8. Запуск проекта



```
PS C:\Users\gavri\OneDrive\Рабочий стол\vebka-43> npm run start
Debugger attached.

> vebka-43@1.0.0 start
> node dist/pract.js

Debugger attached.
{
  id: 5,
  company: 'Corporation',
  isPublished: true,
  tags: [ 'TypeScript', 'JavaScript' ],
  x: 10
}
No age of the user
Hello from User.greet()
hello world
ID is equal to ID 123
ID is equal to 123
Contract signed by Anna, email: anna@gmail.com, creditscore: 800
User not authorized
No user was found with that username
Wrong credentials
Internal error
Anna
56
{
  employeeId: 1,
  name: 'Jessica',
  startDate: 2026-05-05T04:29:52.379Z,
  department: 'Finance'
}
Waiting for the debugger to disconnect...
```

Рисунок 8-запуск проекта

Ссылка на GitHub:

[https://github.com/NoNeme-netizen/vebka\\_43.git](https://github.com/NoNeme-netizen/vebka_43.git)